

8

OTHER ACCESS CHECKING USE CASES



Access checks determine what access a caller should have when opening a kernel resource. However, we sometimes perform them for other reasons, as they can serve as additional security checks. This chapter details some examples of using access checks as a secondary security mechanism.

We'll start by looking at traversal checking, which determines whether a caller has access to a hierarchy of resources. Next, we'll discuss how access checks are used when a handle is duplicated. We'll also consider how an access check can limit access to kernel information, such as process listings, from sandboxed applications. Finally, I'll describe some additional PowerShell commands that automate the access checking of resources.

Traversal Checking

When accessing a hierarchical set of resources, such as an object directory tree, the user must traverse the hierarchy until they reach the target resource. For every directory or container in the hierarchy, the system performs an access check to determine whether the caller can proceed to

the next container. This check is called a *traversal check*, and it's performed whenever code looks up a path inside the I/O manager or object manager. For example, **Figure 8-1** shows the traversal checks needed to access an OMNS object using the path `ABC\QRS\XYZ\OBJ`.

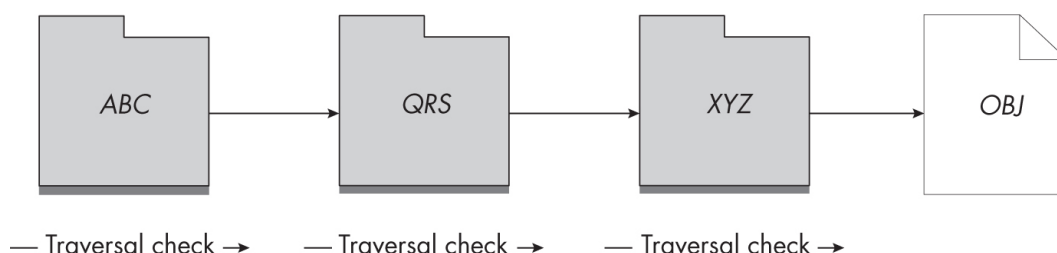


Figure 8-1: Traversal checks required to access `OBJ`

As you can see, three access checks must be performed before we can access `OBJ`. Each access check extracts the security descriptor from the container and then checks the type-specific access to see if traversal is allowed. Both the OMNS and file directories can grant or deny `Traverse` access. If, for example, `QRS` denied `Traverse` access to the caller, the traversal check would fail, as shown in **Figure 8-2**.

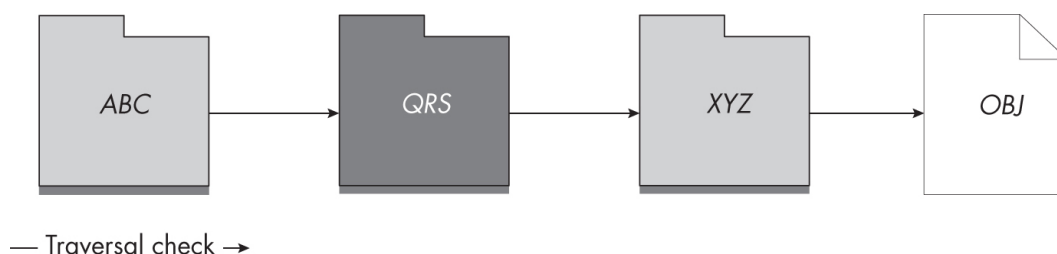


Figure 8-2: Traversal checks blocked at `QRS`

Even if the caller would pass the access checks for `XYZ` and `OBJ`, because `QRS` now denies access via the traversal check, it's no longer possible for them to access `OBJ` using the `ABC\QRS\XYZ\OBJ` path.

The traversal check prevents a user from accessing their resources if any parent container denies `Traverse` access. This is unexpected behavior—why shouldn't a user be able to access their own resources? It also introduces a performance concern. If a user must have access to every parent container to access their files, then the kernel must expend time and effort performing an access check for each container, when all that matters

security-wise is whether the user has access to the resource they want to open.

The SeChangeNotifyPrivilege Privilege

To make the traversal check behavior closer to how you might expect it to work and reduce the performance impact, the SRM defines the `SeChangeNotifyPrivilege` privilege, which almost every `Token` object has enabled by default. When this privilege is enabled, the system bypasses the entire traversal check and lets users access resources that an inaccessible parent would otherwise block. In [Listing 8-1](#), we verify the privilege's behavior using OMNS directory objects.

```
PS> $path = "\BaseNamedObjects\ABC\QRS\XYZ\OBJ"
❶ PS> $os = New-NtMutant $path -CreateDirectories
❷ PS> Enable-NtTokenPrivilege SeChangeNotifyPrivilege
PS> Test-NtObject $path
True

PS> $sd = New-NtSecurityDescriptor -EmptyDacl
❸ PS> Set-NtSecurityDescriptor "\BaseNamedObjects\ABC\QRS" $sd Dacl
PS> Test-NtObject $path
❹ True

❺ PS> Disable-NtTokenPrivilege SeChangeNotifyPrivilege
PS> Test-NtObject $path
False

❻ PS> Test-NtObject "OBJ" -Root $os[1]
True
```

Listing 8-1: Testing SeChangeNotifyPrivilege to bypass traversal checks

We first create a `Mutant` object and all its parent directories, automating the directory creation by using the `CreateDirectories` property ❶. We ensure the privilege is enabled ❷ and then use the `Test-NtObject` command to check whether the `Mutant` object can be opened. In the output, we can see we're able to open the `Mutant` object.

We then set a security descriptor with an empty DACL on the *QRS* directory ❸. This should block all access to the directory object, including `Traverse` access. But when we check our access again, we see that we can still access the `Mutant` object because we have the `SeChangeNotifyPrivilege` privilege enabled ❹.

We now disable the privilege and try again to open the `Mutant` object ❺. This time, the directory traversal fails. Without the `SeChangeNotifyPrivilege` privilege or access to the *QRS* directory, we can no longer open the `Mutant` object. However, our final check demonstrates that if we have access to a parent after *QRS*, such as *XYZ*, we can access the `Mutant` object via a relative open by using the directory as the `Root` parameter ❻.

Limited Checks

The kernel contains an additional performance improvement for traversal checks. If the `SeChangeNotifyPrivilege` privilege is disabled, the kernel will call the `SeFastTraverseCheck` function, which performs a more limited check instead of a full access check. For completeness, I have reimplemented the `SeFastTraverseCheck` function in PowerShell so that we can explore its behavior in more detail. [Listing 8-2](#) shows the implementation.

```
function Get-FastTraverseCheck {  
    Param(  
        ❶ $TokenFlags,  
        $SecurityDescriptor,  
        $AccessMask  
    )  
  
    ❷ if ($SecurityDescriptor.DaclNull) {  
        return $true  
    }  
  
    ❸ if (($TokenFlags -band "IsFiltered, IsRestricted") -ne 0) {  
        return $false  
    }  
  
    $sid = Get-Ntsid -KnownSid World  
    foreach($ace in $SecurityDescriptor.Dacl) {  
        ❹ if ($ace.IsInheritedOnly -or !$ace.IsAccessGranted($AccessMask)) {  
            continue  
        }  
    }  
}
```

```
    }  
    ❸ if ($ace.IsDeniedAce) {  
        return $false  
    }  
    ❹ if ($ace.IsAllowedAce -and $ace.Sid -eq $sid) {  
        return $true  
    }  
    }  
    ❺ return $false  
}
```

Listing 8-2: A PowerShell implementation of `SeFastTraverseCheck`

First, we define the three parameters the function takes: the token's flags, a directory object's security descriptor, and the `Traverse` access rights to check ❶. We specify the access rights because the object manager and the I/O manager use this function for `Directory` and `File` objects, and the value of the `Traverse` access right differs between the two object types; specifying the access as a parameter allows the check function to handle both cases.

Next, we check whether the security descriptor's DACL is `NULL`, granting access if it is ❷. We follow this with a check on two token flags ❸. If the flags indicate that the token is filtered or restricted, then the fast check fails. The kernel copies these flags from the caller's `Token` object. We can get the flags from user mode using the `Flags` property on a `Token` object, as shown in [Listing 8-3](#).

```
PS> $token = Get-NtToken -Pseudo -Primary  
PS> $token.Flags  
VirtualizeAllowed, IsFiltered, NotLow  
  
PS> $token.ElevationType  
Limited
```

Listing 8-3: Querying token flags

Notice that the flags include `IsFiltered`. If you're not running in a restricted token sandbox, why would this flag be set? Querying the token elevation type shows that it's `Limited`, which means it's the default token for a UAC administrator. To convert the full administrator token to the default token, LSASS uses the `NtFilterToken` system, which will set the `IsFiltered` flag but not `IsRestricted`, as it's only removing groups, not adding restricted SIDs. This means that while a UAC admin running code as the default user can never pass the fast traversal check, a normal user could. This behavior doesn't have any security implication, but it does mean that if `SeChangeNotifyPrivilege` is disabled, resource lookup performance will suffer.

The final check in [Listing 8-3](#) consists of enumerating the DACL's ACEs. If the ACE is inherit-only or doesn't contain the required `Traverse` access mask, it's skipped ❹. If it's a `Denied` ACE, the fast traverse check fails ❺, and the ACE's SID is not checked at all. Finally, if the ACE is an `Allowed` ACE and the SID equals the *Everyone* group's SID, the fast check succeeds ❻. If there are no more ACEs, the check fails ❼.

Note that this fast check doesn't consider whether the caller's token has the *Everyone* group enabled. This is because typically the only way to remove the *Everyone* group would be to filter the token. The big exception to this is the anonymous token, which doesn't have any groups but is also not filtered in any way.

Now let's turn to another use for the access check: considering the granted access when assigning a duplicated handle.

Handle Duplication Access Checks

The system always performs an access check when creating or opening a kernel resource that returns a handle. But what about when that handle is duplicated? In the simplest case, when the new handle has the same granted access mask as the original one, the system won't perform any checks. It's also possible to drop some parts of the granted access mask, and doing so won't trigger an additional access check either. However, if you want to add additional access rights to the duplicated handle, the ker-

nel will query the security descriptor from the object and perform a new access check to determine whether to allow the access.

When you duplicate a handle, you must specify both the source and destination process handles, and the access check occurs in the context of the destination process. This means the access check considers the destination process's primary token, not the source process's, which could be an issue if a privileged process tried to duplicate a handle to a less privileged process with additional access. Such an operation would fail with `Access Denied`.

Listing 8-4 demonstrates this handle duplication access check behavior.

```
PS> $sd = New-NtSecurityDescriptor -EmptyDacl
❶ PS> $m = New-NtMutant -Access ModifyState, ReadControl -SecurityDescriptor $sd
❷ PS> Use-NtObject($m2 = Copy-NtObject -Object $m) {
    $m2.GrantedAccess
}
ModifyState, ReadControl

PS> $mask = Get-NtAccessMask -MutantAccess ModifyState
❸ PS> Use-NtObject($m2 = Copy-NtObject -Object $m -DesiredAccessMask $mask) {
    $m2.GrantedAccess
}
ModifyState

❹ PS> Use-NtObject($m2 = Copy-NtObject -Object $m -DesiredAccess GenericAll) {
    $m2.GrantedAccess
}
Copy-NtObject : (0xC0000022) - {Access Denied}
A process has requested access to an object, ...
```

Listing 8-4: Testing the handle duplication access check behavior

We first create a new `Mutant` object with an empty DACL and request only `ModifyState` and `ReadControl` access on the handle ❶. This will block all users from accessing the object, except for the owner, who can be granted `ReadControl` and `WriteDac` access thanks to the owner check described in the

previous chapter. We test the duplication by requesting the same access, which the new handle returns ❷.

Next, we request `ModifyState` access only ❸. As the `Mutant`'s DACL is empty, this access right wouldn't be granted during an access check, and because we get `ModifyState` on the new handle, we know that no access check took place. Finally, we try to increase our access by requesting `GenericAll` access ❹. An access check must now take place, as we're requesting greater access rights than the handle currently has. This check results in an `Access Denied` error.

If we hadn't set a security descriptor when creating the `Mutant`, there would be no security associated with the object, and this last check would have succeeded, granting `Full Access`. As mentioned in [Chapter 2](#), you need to be careful when duplicating unnamed handles to less privileged processes if you're dropping access; the destination process might be able to reduplicate the handle to one with more access. In [Listing 8-5](#), we test the `NtDuplicateObject NoRightsUpgrade` flag to see how it affects handle duplication access checking.

```
PS> $m = New-NtMutant -Access ModifyState
PS> Use-NtObject($m2 = Copy-NtObject -Object $m -DesiredAccess GenericAll) {
    $m2.GrantedAccess
}
ModifyState, Delete, ReadControl, WriteDac, WriteOwner, Synchronize

PS> Use-NtObject($m2 = Copy-NtObject -Object $m -NoRightsUpgrade) {
    Use-NtObject($m3 = Copy-NtObject -Object $m2 -DesiredAccess GenericAll) {}
}
Copy-NtObject : (0xC0000022) - {Access Denied}
A process has requested access to an object, ...
```

Listing 8-5: Testing the `NtDuplicateObject NoRightsUpgrade` flag

We start by creating an unnamed `Mutant` object, which will have no associated security descriptor. We request the initial handle with `ModifyState` access only. However, our attempt to duplicate a new handle with `GenericAll` access succeeds, granting us complete access.

Now we test the `NoRightsUpgrade` flag. Because we don't specify any access mask, the handle will be duplicated with `ModifyState` access. With the new handle, we then perform another duplication, this time requesting `GenericAll` access. We can observe that the handle duplication fails. This isn't due to an access check; instead, it's because of a flag set on the handle entry in the kernel indicating that any request for more access should fail immediately. This prevents the handle from being used to gain additional access rights.

The incorrect handling of duplicate handles can lead to vulnerabilities; for example, CVE-2019-0943, an issue I discovered in a privileged service responsible for caching the details of font files on Windows. The service duplicated a `Section` object handle to a sandbox process with read-only access. However, the sandbox process could convert the handle back to a writable section handle, and the section could be mapped into memory as writable. This allowed the sandbox process to modify the state of the privileged service and escape the sandbox. Windows fixed the vulnerability by duplicating the handle using the `NoRightsUpgrade` flag.

THE THREAD PROCESS CONTEXT

Every thread is associated with a process. Normally, when an access check occurs, the kernel extracts the `Process` object from the calling thread's object structure and uses it to look up the primary token for the access check. But the thread has a second `Process` object associated with it: the current *process context*, which indicates the process in which the thread is currently executing code.

Normally, these objects are the same; however, the kernel sometimes switches the current process context to another process to save time during certain tasks, such as handle or virtual memory access. When a process switch has occurred, any access check on the thread will look up the primary token of the switched-to process rather than the token belonging to the process associated with the thread. Handle duplication operations make use of this process context switch: the kernel first queries the source process's handle table, then switches

the process context for the calling thread to the destination process to create the new handle in that process's handle table.

A process can abuse this behavior to duplicate a handle with more access to a less privileged process. If you call the `NtDuplicateObject` system call while impersonating your own token with access to the object, when the access check runs it will capture the `SECURITY_SUBJECT_CONTEXT` for the thread, setting the primary token for the destination process. Crucially, though, it also sets the impersonation token to the identity being impersonated. The result is that the access check will run against the caller's impersonation token rather than the destination process's primary token. This allows a handle to be duplicated with additional granted access rights even if the destination process's primary token could not pass an access check for those rights. You probably shouldn't rely on this behavior in practice, though; it's an implementation detail and might be subject to change.

The access checks that occur during traversal checking and handle duplication are typically hidden from view, but both relate to the security of an individual resource. Next, we'll discuss how access checks limit the information we can extract and the operations we can perform for a group of resources. These restrictions occur based on the caller's token, regardless of the individual access set for those resources.

Sandbox Token Checks

Beginning in Windows 8, Microsoft has tried to make it harder to compromise the system by escaping sandbox token restrictions. This is especially important for software such as web browsers and document readers, which process untrusted content from the internet.

The kernel implements two APIs that use an access check to determine whether the caller is in a sandbox: `ExIsRestrictedCaller`, introduced in Windows 8, and `RtlIsSandboxToken`, introduced in Windows 10. These APIs produce equivalent results; the difference between them is that

`ExIsRestrictedCaller` checks the token of the caller, while `RtlIsSandboxToken` checks a specified `Token` object that doesn't have to be the caller's.

Internally, these APIs perform an access check for the token and grant access only if the token is not in a sandbox. [Listing 8-6](#) shows a reimplementation of this access check in PowerShell.

```
PS> $type = New-NtType -Name "Sandbox" -GenericRead 0x20000
           -GenericAll 0x1F0001
PS> $sd = New-NtSecurityDescriptor -NullDacl -Owner "SY" -Group "SY"
           -Type $type
PS> Set-NtSecurityDescriptorIntegrityLevel $sd Medium -Policy NoReadUp
PS> Get-NtGrantedAccess -SecurityDescriptor $sd -Access 0x20000 -PassResult
Status                Granted Access Privileges
-----
STATUS_SUCCESS        GenericRead      NONE

PS> Use-NtObject($token = Get-NtToken -Duplicate -IntegrityLevel Low) {
    Get-NtGrantedAccess -SecurityDescriptor $sd -Access 0x20000
    -Token $token -PassResult
}
Status                Granted Access Privileges
-----
STATUS_ACCESS_DENIED  None              NONE
```

Listing 8-6: An access check for a sandbox token

First, we need to define a dummy kernel object type using the `New-NtType` command. This allows us to specify the generic mapping for the access check. We specify only the `GenericRead` and `GenericAll` values, as write and execute access are not important in this context. Note that the new type is local to PowerShell; the kernel doesn't know anything about it.

We then define a security descriptor with a NULL DACL and the owner and group SIDs set to the `SYSTEM` user. The use of a NULL DACL will deny access to lowbox tokens, as described in the previous chapter, but not to any other sandbox token type, such as restricted tokens.

To handle other token types, we add a `Medium` mandatory label ACE with a `NoReadUp` policy. As a result, any token with an integrity level lower than `Medium` will be denied access to the mask specified in the generic mapping's `GenericRead` field. Lowbox tokens ignore the `Medium` mandatory label, but we've covered these tokens using the NULL DACL. Note that this security descriptor doesn't consider restricted tokens with a `Medium` integrity level to be sandbox tokens. It's not clear if this is an intentional oversight or a bug in the implementation.

We can now perform an access check with the `Get-NtGrantedAccess` command, using the current, non-sandboxed token. The access check succeeds, granting us `GenericRead` access. If we repeat the check with a token that has a `Low` integrity level, the system denies us access, indicating that the token is sandboxed.

Behind the scenes, the kernel APIs call the `SeAccessCheck` API, which will return an error if the caller has an Identification-level impersonation token. Therefore, the kernel will consider some impersonation tokens to be sandboxed even if the implementation in [Listing 8-6](#) would indicate otherwise.

When either API indicates that the caller is sandboxed, the kernel changes its behavior to do the following:

- List only processes and threads that can be directly accessed.
- Block access to loaded kernel modules.
- Enumerate open handles and their kernel object addresses.
- Create arbitrary file and object manager symbolic links.
- Create a new restricted token with more access.

For example, in [Listing 8-7](#), we query for handles while impersonating a `Low` integrity level token and are denied access.

```
PS> Invoke-NtToken -Current -IntegrityLevel Low {  
    Get-NtHandle -ProcessId $pid  
}
```

```
Get-NtHandle : (0xC0000022) - {Access Denied}
A process has requested access to an object, ...
```

Listing 8-7: Querying for handle information while impersonating a Low integrity level token

While only kernel-mode code can access `ExIsRestrictedCaller`, you can access `RtlIsSandboxToken` in user mode, as it's also exported in `NTDLL`. This allows you to query the kernel using a token handle to find out whether the kernel thinks it is a sandbox token. The `RtlIsSandboxToken` API exposes its result in the `Token` object's `IsSandbox` property, as shown in [Listing 8-8](#).

```
PS> Use-NtObject($token = Get-NtToken) {
    $token.IsSandbox
}
False

PS> Use-NtObject($token = Get-NtToken -Duplicate -IntegrityLevel Low) {
    $token.IsSandbox
}
True
```

Listing 8-8: Checking the sandbox status of tokens

The `Process` object returned by `Get-NtProcess` has an `IsSandboxToken` property. Internally, this property opens the process's token and calls `IsSandbox`. We can use this property to easily discover which processes are sandboxed, by using the script in [Listing 8-9](#), for example.

```
PS> Use-NtObject($ps = Get-NtProcess -FilterScript {$_.IsSandboxToken}) {
    $ps | ForEach-Object {Write-Host "$($_.ProcessId) "$($_.Name)}
}
7128 StartMenuExperienceHost.exe
7584 TextInputHost.exe
4928 SearchApp.exe
7732 ShellExperienceHost.exe
1072 Microsoft.Photos.exe
7992 YourPhone.exe
```

Listing 8-9: Enumerating all sandboxed processes for the current user

These sandbox checks are an important feature for limiting information disclosure and restricting dangerous functionality such as symbolic links, which improve an attacker's chances of escaping the sandbox and gaining additional privileges. For example, blocking access to the handle table prevents the disclosure of kernel object addresses that could be used to exploit kernel memory corruption vulnerabilities.

We've now covered three uses of the access check for purposes not related to opening a resource. We'll finish this chapter by describing some commands that simplify access checking over a range of individual resources.

Automating Access Checks

The previous chapter provided a worked example that used `Get-NtGrantedAccess` to determine the granted access for a collection of kernel objects. If you want to check a different type of resource, such as files, you'll need to modify that script to use file commands.

Because checking for the granted access across a range of resources is such a useful operation, the PowerShell module comes with several commands to automate the process. The commands are designed to allow you to quickly assess the security attack surface of available resources on a Windows system. They all start with `Get-Accessible`, and you can use `Get-Command` to list them, as shown in **[Listing 8-10](#)**.

```
PS> Get-Command Get-Accessible* | Format-Wide
Get-AccessibleAlpcPort           Get-AccessibleDevice
Get-AccessibleEventTrace        Get-AccessibleFile
Get-AccessibleHandle            Get-AccessibleKey
Get-AccessibleNamedPipe         Get-AccessibleObject
Get-AccessibleProcess           Get-AccessibleScheduledTask
Get-AccessibleService           Get-AccessibleToken
Get-AccessibleWindowStation     Get-AccessibleWnf
```

Listing 8-10: Listing the `Get-Accessible*` commands

We'll come back to some of these commands in later chapters. Here, we'll focus on the `Get-AccessibleObject` command, which we can use to automate access checking over the entire OMNS. The command lets you specify an OMNS path to check, then enumerates the OMNS and reports either the maximum granted access or whether a specific access mask can be granted.

You can also specify what tokens to use for the access check. The command can source tokens from the following list:

- Token objects
- Process objects
- Process names
- Process IDs
- Process command lines

If you specify no options when running the command, it will use the current primary token. It will then enumerate all objects based on an OMNS path and perform an access check for every token specified. If the access check succeeds, then the command generates a structured object containing the details of the result. [Listing 8-11](#) shows an example.

```
PS> Get-AccessibleObject -Path "\"
TokenId Access                               Name
-----
C5856B9 GenericExecute|GenericRead          \
```

Listing 8-11: Getting accessible objects from the OMNS root

Here, we run the command against the root of the OMNS, and we receive three columns in the output:

TokenId The unique identifier of the token used for the access check

Access The granted access, mapped to generic access rights

Name The name of the checked resource

We can use the `TokenId` to distinguish the results for the different tokens specified to the command.

This output is only a subset of the result produced by the `Get-AccessibleObject` command. You can extract the rest of the information using commands like `Format-List`. You can also display the copy of the security descriptor used to perform the access check with the `Format-NtSecurityDescriptor` PowerShell command, as shown in [Listing 8-12](#).

```
PS> Get-AccessibleObject -Path \ | Format-NtSecurityDescriptor -Summary
<Owner> : BUILTIN\Administrators
<Group> : NT AUTHORITY\SYSTEM
<DACL>
Everyone: (Allowed)(None)(Query|Traverse|ReadControl)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
BUILTIN\Administrators: (Allowed)(None)(Full Access)
NT AUTHORITY\RESTRICTED: (Allowed)(None)(Query|Traverse|ReadControl)
```

Listing 8-12: Displaying the security descriptor used for the access check

As we've run the command against a directory here, you might wonder if it will also list the objects contained within the directory. By default, no; the command opens the path as an object and does an access check. If you want to recursively check all objects in the directory, you need to specify the `Recurse` parameter. The `Get-AccessibleObject` command also accepts a `Depth` parameter you can use to specify the maximum recursive depth. If you run a recursive check as a non-administrator user, you might see a lot of warnings, as in [Listing 8-13](#).

```
PS> Get-AccessibleObject -Path "\" -Recurse
WARNING: Couldn't access \PendingRenameMutex - Status: STATUS_ACCESS_DENIED
WARNING: Couldn't access \ObjectTypes - Status: STATUS_ACCESS_DENIED
--snip--
```

Listing 8-13: Warnings when recursively enumerating objects

You can turn off the warnings by setting the `WarningAction` parameter to `Ignore`, but keep in mind that they're trying to tell you something. For the

command to work, it needs to open each object and query its security descriptor. From user mode, this requires passing the access check during the opening; if you don't have permission to open an object for `ReadControl` access, the command can't perform an access check. For better results, you can run the command as an administrator, and for the best results, run it as the *SYSTEM* user by using the `Start-Win32ChildProcess` command to start a *SYSTEM* PowerShell shell.

By default, the command will perform the access check using the caller's token. But if you're running the command as an administrator, you probably won't want this behavior, as almost all resources will allow administrators full access. Instead, consider specifying arbitrary tokens to check against the resource. For example, when run as a UAC administrator, the following command recursively opens the resources using the administrator token but performs the access check with the non-administrator token from the Explorer process:

```
PS> Get-AccessibleObject -Path \ -ProcessName explorer.exe -Recurse
```

It's common to want to filter the list of objects to check. You could run the access check against all the objects and then filter the list afterward, but this would require a lot of work that you'll then just throw away. To save you some time, the `Get-AccessibleObject` command supports multiple filter parameters:

TypeFilter A list of NT type names to check

Filter A name filter used to restrict which objects are opened; can contain wildcards

Include A name filter used to determine which results to include in the output

Exclude A name filter used to determine which results to exclude from the output

Access An access mask used to limit the output to only objects with specific granted access

For example, the following command will find all the `Mutant` objects that can be opened with `GenericAll` access:

```
PS> Get-AccessibleObject -Path \ -TypeFilter Mutant -Access GenericAll -Recurse
```

By default, the `Access` parameter requires that all access be granted before outputting a result. You can modify this by specifying `AllowPartialAccess`, which will output any result that partially matches the specified access. If you want to see all results regardless of the granted access, specify `AllowEmptyAccess`.

Worked Examples

Let's wrap up with some worked examples that use the commands you've learned about in this chapter.

Simplifying an Access Check for an Object

In the previous chapter, we used the `Get-NtGrantedAccess` command to automate an access check against kernel objects and determine their maximum granted access. To accomplish this, we first needed to query for an object's security descriptor. We then passed this value to the command along with the type of kernel object to check.

If you have a handle to an object, you can simplify the call to the `Get-NtGrantedAccess` command by specifying the object with the `Object` parameter, as shown in [Listing 8-14](#).

```
PS> $key = Get-NtKey HKLM\Software -Win32Path -Access ReadControl
PS> Get-NtGrantedAccess -Object $key
QueryValue, EnumerateSubKeys, Notify, ReadControl
```

Listing 8-14: Running an access check on an object

Using the `object` parameter eliminates the need to manually extract the security descriptor from the object and will automatically select the correct generic mapping structure for the kernel object type. This reduces the risk that you'll make mistakes when performing an object access check.

Finding Writable Section Objects

The system uses `Section` objects to share memory between processes. If a privileged process sets a weak security descriptor, it might be possible for a less privileged process to open and modify the contents of the section. This can lead to security issues if that section contains trusted parameters that can trick the privileged process into performing privileged operations.

I identified a vulnerability of this class, CVE-2014-6349, in Internet Explorer's sandbox configuration. The configuration incorrectly secured a shared `Section` object, allowing sandboxed Internet Explorer processes to open it and disable the sandbox entirely. I discovered this issue by performing an access check for `MapWrite` access on all named `Section` objects. Once I had identified all sections with this access right, I manually determined whether any of them were exploitable from the sandbox. In

Listing 8-15, we automate the discovery of writable sections using the `Get-AccessibleObject` command.

```
❶ PS> $access = Get-NtAccessMask -SectionAccess MapWrite -AsGenericAccess
❷ PS> $objs = Use-NtObject($token = Get-NtToken -Duplicate
    -IntegrityLevel Low) {
    ❸ Get-AccessibleObject -Win32Path "\" -Recurse -Token $token
    -TypeFilter Section -Access $access
    }
PS> $objs | ForEach-Object {
    ❹ Use-NtObject($sect = Get-NtSection -Path $_.Name) {
        Use-NtObject($map = Add-NtSection $sect -Protection ReadWrite
        -ViewSize 4096) {
            Write-Host "$($sect.FullPath)"
            Out-HexDump -ShowHeader -ShowAscii -HideRepeating -Buffer $map |
            Out-Host
        }
    }
}
```

```

}
\Sessions\1\BaseNamedObjects\windows_ie_global_counters
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F - 0123456789ABCDEF
-----
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 - .....
-> REPEATED 1 LINES
00 00 00 00 00 00 00 00 00 00 00 00 1C 00 00 00 - .....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 - .....
--snip--

```

Listing 8-15: Enumerating writable Section objects for a Low integrity level token

We start by calculating the access mask for the `MapWrite` access and converting it into a generic access enumeration ❶. The `Get-AccessibleObject` command takes only generic access, as it doesn't know ahead of time what objects you're likely to want to check for. We then duplicate the current user's token and set its integrity level to `Low`, producing a simple sandbox ❷.

We pass the token and access mask to `Get-AccessibleObject`, performing a recursive check in the user's *BaseNamedObjects* directory by specifying a single path separator to the `Win32Path` parameter ❸. The results returned from the command should contain only sections that can be opened for `MapWrite` ACCESS.

Finally, we enumerate the list of discovered sections, displaying their names and the initial contents of any discovered writable Section object ❹. We open each named section, map up to the first 4,096 bytes into memory, and then output the contents as a hex dump. We map the section as writable, as it's possible the Section object's security descriptor grants `MapWrite` access but that the section was created read-only. In this case, mapping `ReadWrite` will fail with an error.

You can use this script as is to find noteworthy writable sections. You don't have to use a sandbox token; it can be interesting to see the sections available for a normal user that are owned by privileged processes. You

can also use this as a template for performing the same check for any other kernel object type.

Wrapping Up

In this chapter, we looked at some examples of the uses of access checking outside of opening a resource. We first considered traversal checks, which are used to determine if a user can traverse a hierarchical list of containers, such as object directories. Then we discussed how access checks are used when handles are duplicated between processes, including how this can create security issues if the object has no name or security descriptor configured.

Next, we explored how an access check can be used to determine if a caller's token is sandboxed. The kernel does this to limit access to information or certain operations, to make it more difficult to exploit specific classes of security vulnerabilities. Finally, we saw how to automate access checks for various resource types with `Get-Accessible` commands. We looked at the basic parameters common to all commands and how to use them to enumerate accessible named kernel objects.

That's the end of our examination of the access check process. In the next chapter, we'll cover the last remaining responsibility of the SRM: security auditing.